VECTOR INDIA

(Advanced Embedded System Training Institute)

[Bangaluru, Chennai, Hyderabad]

A PROJECT SYNOPSIS
ON

SMART INDUSTRIAL RISK DETECTION AND EMERGENCY ALERT SYSTEM

An Embedded Industrial Safety Monitoring System Based on the LPC2129 ARM7 Microcontroller

---

SUBMITTED BY
Tanmaye M S (V25BE7T2)

VECTOR INDIA INSTITUTE
(33/49,12th Main Rd,4th T Block East, KV Layout, Jayanagar, Bengaluru, Karnataka 560011)
[July 2026]

INDEX

1. Introduction 3

1.1 Background and Need for the Project 3

1.2 Problem Statement 3

1.3 Objectives of the Project 3

1.4 Scope of the Project 4

2. Components Used 5

2.1 Hardware Components 5

2.2 Software Tools 6

2.3 Communication Interfaces Used 6

3. Working and Block Diagram Explanation 7

3.1 System Block Diagram 7

3.2 Stage-wise Working Principle 7

3.3 Working Flow Diagram 9

4. Code Review 10

4.1 Program Structure Overview 10

4.2 Key Code Modules 10

4.3 Code Analysis and Review Remarks 13

5. Advantages and Disadvantages 14

5.1 Advantages 14

5.2 Disadvantages / Limitations 14

6. Future Scope 15

7. Bibliography 16

CHAPTER 1 | INTRODUCTION

1.1 Background and Need for the Project

Modern manufacturing industries operate hundreds of machines continuously, and these machines are exposed to heat, high electrical current, mechanical vibration, and, in many cases, combustible gases and chemical vapours. Under normal working conditions such machines run safely; however, even a minor fault developing inside a machine can gradually escalate into a major industrial accident if it remains undetected. A motor may begin to overheat due to poor ventilation, a gas pipeline may develop a small leakage, a bearing may generate excessive vibration due to wear, or an electrical circuit may draw abnormal current because of overload. Each of these conditions appears insignificant at first but, if ignored, can result in equipment failure, fire hazards, gas explosions, production loss, and risk to human life.

Human operators cannot continuously monitor every machine and every parameter across a large industrial floor. Since multiple machines run simultaneously, it becomes practically impossible to manually observe temperature changes, gas leakage, vibration levels, and electrical faults in real time. As a result, dangerous conditions may remain unnoticed until they become critical. This creates the need for an automatic, round-the-clock monitoring system that can continuously observe industrial parameters and immediately raise an alert whenever an abnormal condition is detected — without depending on human vigilance.

1.2 Problem Statement

Industries currently rely largely on manual inspection and isolated analogue instruments to track machine health. This approach is slow, inconsistent, and reactive rather than preventive — faults are frequently discovered only after damage has already occurred. There is a clear requirement for an embedded, low-cost, real-time monitoring solution capable of observing multiple safety-critical parameters simultaneously, making an immediate decision when a parameter crosses its safe limit, and triggering the appropriate protective action (visual, audible, wireless, and electro-mechanical) without waiting for a human response.

1.3 Objectives of the Project

- To design and develop an intelligent industrial safety monitoring system built around the LPC2129 ARM7 microcontroller.
- To detect industrial hazards — high temperature, gas leakage, fire, abnormal vibration, current overload, and high humidity — in real time.
- To continuously monitor environmental and machine parameters using a network of dedicated sensors.
- To automatically generate visual, audible, and wireless emergency alerts whenever an unsafe condition is detected.
- To automatically disconnect the affected machine from supply using relay-based protection when a hazard is confirmed.
- To implement industrial communication protocols such as CAN, UART, I2C and SPI for multi-node monitoring and data logging.
- To log every fault event with an accurate time-stamp using an RTC module and retain the history in EEPROM even after a power failure.
- To demonstrate AI-inspired embedded decision-making, where the controller autonomously classifies a situation as safe, warning, or emergency and selects the correct response.

1.4 Scope of the Project

The system is designed for deployment in factories, chemical plants, boiler rooms, manufacturing units and other environments where continuous safety supervision is essential. It is intended as an embedded major-project implementation demonstrating practical, industry-oriented monitoring rather than a full industrial-grade product; nevertheless, the architecture — sensor acquisition, threshold-based decision logic, multi-channel alerting, inter-node communication and non-volatile event logging — mirrors the core building blocks used in real industrial safety controllers. The synopsis stage covers system design, component selection, working methodology, and the embedded logic that will subsequently be implemented and tested on hardware.

CHAPTER 2 | COMPONENTS USED

2.1 Hardware Components

The project uses a combination of sensing, processing, communication and output devices, organised into three functional groups as shown below.

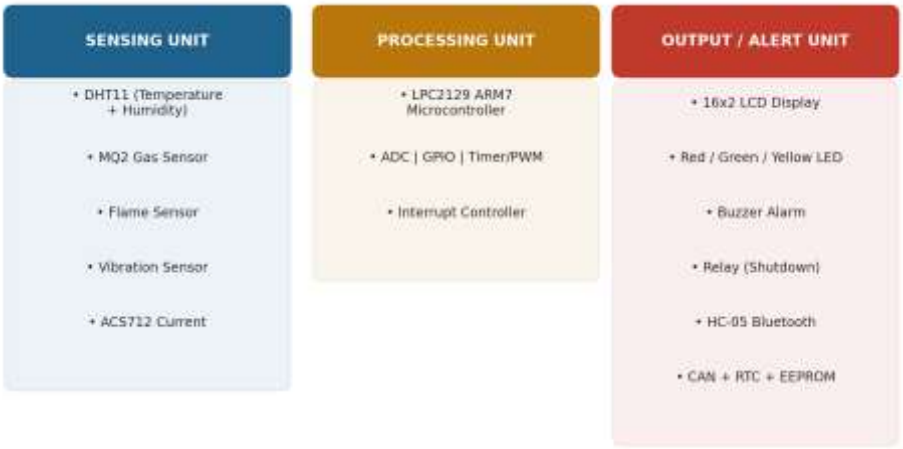


Fig. 2.1 – Component Category Overview

Sl. No.	Component	Function in the Project
1	LPC2129 ARM7 Development Board	Core processing unit; reads all sensors, executes decision logic, drives every output and communication module.
2	DHT11 Temperature & Humidity Sensor	A single digital sensor that provides both machine/ambient temperature and humidity readings over a 1-wire interface, used for overheat detection and humidity monitoring.
3	MQ2 Gas Sensor	Detects combustible/toxic gas concentration for leakage and explosion-risk monitoring.
4	Flame Sensor Module	Detects the infrared signature of a flame for fast, interrupt-driven fire detection.
5	Vibration Sensor	Detects abnormal mechanical vibration caused by bearing wear or mechanical imbalance.
6	ACS712 Current Sensor	Measures load current to detect an overload/short-circuit condition on the machine.
7	16×2 LCD Display	Displays system status and the specific alert / warning message locally.
8	HC-05 Bluetooth Module	Wirelessly transmits emergency alert notifications to a nearby mobile device.

Sl. No.	Component	Function in the Project
9	CAN Transceiver (MCP2551)	Broadcasts emergency information to other industrial monitoring nodes over the CAN bus.
10	RTC Module (DS1307)	Time-stamps every fault/emergency event for accurate historical logging.
11	EEPROM (24C256) / SD Card Module	Provides non-volatile storage for fault history, retained even after power loss.
12	Relay Module	Electrically disconnects the monitored machine when an emergency is confirmed.
13	Red / Green / Yellow LEDs	Provide immediate visual indication of normal, warning and emergency states.
14	Buzzer	Provides an audible alarm during warning and emergency conditions.
15	Regulated 5V DC Power Supply	Powers the microcontroller board and all connected peripherals.
16	Breadboard and Jumper Wires	Used for prototype-stage interconnection of all modules.

2.2 Software Tools

- Keil µVision — IDE used for writing, compiling and debugging the Embedded C firmware for the LPC2129.
- Flash Magic — Utility used to program (flash) the compiled hex file onto the LPC2129 through its serial bootloader.
- Embedded C — Programming language used to implement sensor acquisition, decision logic and peripheral control.
- CAN Utility Tools — Used to monitor and verify CAN bus messages exchanged between industrial nodes during testing.

2.3 Communication Interfaces Used

Interface	Purpose in this Project
GPIO	Direct digital control of LEDs, buzzer, relay, flame-sensor interrupt input, and 1-wire timing for the DHT11 sensor.
ADC	Converts analog outputs of the MQ2, vibration and ACS712 sensors into digital readings.
UART	Serial communication with the HC-05 Bluetooth module for wireless alerting.
I2C	Communication with the RTC (DS1307) and EEPROM (24C256) for logging.
SPI	High-speed interface used for SD-card-based data logging.
CAN	Real-time broadcast of emergency messages between multiple industrial monitoring nodes.
Interrupts	Immediate, non-polled response to the flame sensor for fast fire detection.

CHAPTER 3 | WORKING AND BLOCK DIAGRAM EXPLANATION

3.1 System Block Diagram

The overall architecture of the system is organised into four functional blocks — the sensor input section, the signal-conditioning section, the LPC2129 processing core, and the communication/output section, as illustrated below.

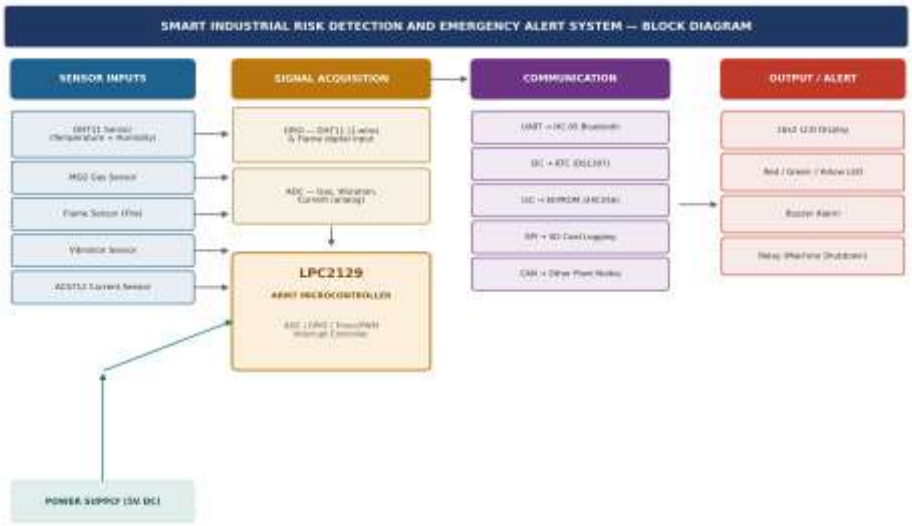


Fig. 3.1 – AI Industrial Risk Detection and Emergency Alert System: Block Diagram

Each analog sensor output (gas, vibration, current) first passes through a signal-conditioning stage before reaching the LPC2129's ADC, while the DHT11 sensor and the flame sensor communicate with the controller directly over digital GPIO lines. The microcontroller reads every channel, evaluates it against predefined safety thresholds, and drives the communication block (UART, I2C, SPI, CAN) and the output block (LCD, LEDs, buzzer, relay) accordingly.

3.2 Stage-wise Working Principle

The complete operation of the system can be described as thirteen functional stages, summarised in Table 3.1.

Stage	Name	Description
1	System Initialization	GPIO, LCD, ADC, UART, I2C, SPI and CAN are initialised; relay stays ON; LCD shows “INDUSTRIAL SAFETY SYSTEM”.
2	Sensor Monitoring	Temperature and humidity are read digitally from the DHT11; gas, vibration and current sensors are sampled continuously through the ADC.
3	Normal Condition	If all readings are within safe limits, the Green LED is ON, buzzer is OFF, relay stays ON; LCD shows “SYSTEM NORMAL”.
4	High Temperature Detection	Red LED ON, buzzer activates, relay disconnects the machine; LCD shows “HIGH TEMP ALERT” and a Bluetooth warning is sent.
5	Gas Leakage Detection	Red LED blinks, buzzer sounds continuously, relay disconnects load; LCD shows “GAS LEAKAGE DETECTED” with an emergency Bluetooth alert.

Stage	Name	Description
6	Fire Detection (Interrupt-driven)	External interrupt fires instantly; alarm and rapid LED blink activate; relay turns OFF; a CAN emergency broadcast is sent to all nodes.
7	Machine Vibration Detection	Yellow warning LED and buzzer activate; LCD shows “MACHINE VIBRATION ALERT” to flag developing mechanical faults.
8	Current Overload Detection	Relay disconnects the load and the alarm activates; LCD shows “OVER CURRENT ALERT” to protect equipment.
9	Humidity Monitoring	Yellow LED activates and LCD shows “HIGH HUMIDITY DETECTED”, useful for storage and chemical-industry areas.
10	CAN Bus Communication	Dedicated nodes (temperature, gas, fire, etc.) exchange emergency information over the CAN bus for plant-wide awareness.
11	RTC Logging (I2C)	Every alert is time-stamped by the DS1307 RTC, e.g. “10:45 AM — GAS ALERT DETECTED”.
12	Data Logging (SPI)	Sensor readings and emergency events are logged to an SD card for later analysis and report generation.
13	Smart Decision-Making	The controller continuously compares live readings with threshold limits and classifies the state as Safe, Warning or Emergency before choosing a response.

3.3 Working Flow Diagram

The stage-wise logic described above can be represented as a continuous decision loop, shown in Fig. 3.2. The controller cycles through sensor acquisition, threshold comparison, condition classification and automatic protective action, while simultaneously logging every event over CAN, RTC and EEPROM/SD-card storage.

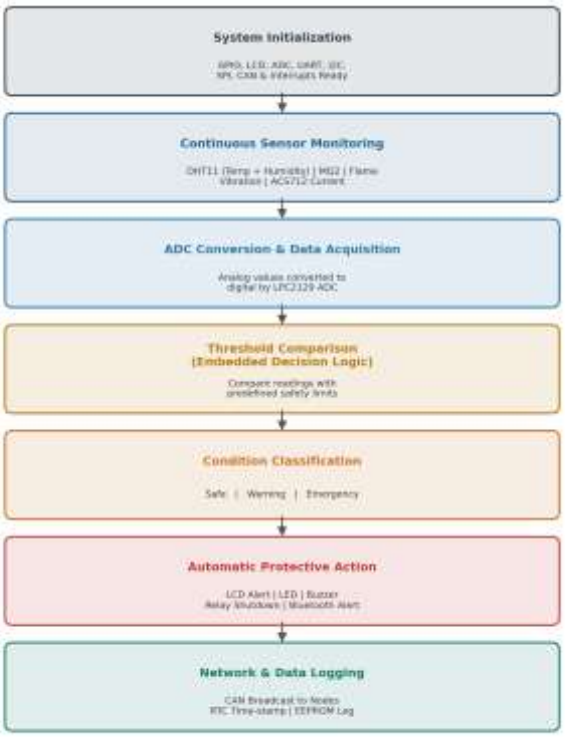


Fig. 3.2 – Working Flow of the Embedded Decision Logic

CHAPTER 4 | CODE REVIEW

4.1 Program Structure Overview

The firmware is developed in Embedded C using Keil µVision and follows a modular structure so that each subsystem — initialisation, sensor acquisition, decision logic, alerting, and communication — is implemented as an independent, testable function. This keeps the main loop compact and makes the code easy to extend when additional sensors or nodes are added. The representative code segments below illustrate the structure and logic that the synopsis design is built around; they will form the basis of the final implementation and are refined further during coding and hardware testing.

4.2 Key Code Modules

(a) Main Program and Global Declarations

```
#include <LPC21xx.H>
#include "lcd.h"
#include "adc.h"
#include "dht11.h"
#include "uart.h"
#include "i2c_rtc.h"
#include "can.h"

#define TEMP_LIMIT      45      // deg C
#define GAS_LIMIT       400     // ADC threshold
#define CURRENT_LIMIT   5000    // mA
#define VIB_LIMIT       600     // ADC threshold
#define HUMIDITY_LIMIT  70      // %RH

unsigned int temp, humidity;      // both read from DHT11
unsigned int gas, vib, current;   // read via ADC

void System_Init(void);
void Sensor_Read(void);
void Evaluate_Conditions(void);
void Raise_Alert(char *msg, unsigned char level);
__irq void Flame_ISR(void);

int main(void)
{
    System_Init();
    LCD_Print("INDUSTRIAL SAFETY");

    while (1)
    {
        Sensor_Read();
        Evaluate_Conditions();
    }
}
```

The main loop keeps the design deliberately simple: initialise once, then continuously read sensors and evaluate conditions. All threshold values are defined as named constants at the top of the file, which makes calibration straightforward without touching the control logic.

(b) System Initialisation

```
void System_Init(void)
{
    PINSEL0 = 0x00050000;      // configure UART0, ADC pins
    IODIR0 |= (LED_MASK | BUZZER | RELAY);
    IOSET0 |= RELAY;           // relay ON (machine running) at start

    LCD_Init();
    ADC_Init();                // gas, vibration, current channels
    DHT11_Init();              // single-wire GPIO for temp + humidity
    UART0_Init(9600);          // HC-05 Bluetooth link
    I2C_Init();                // RTC + EEPROM
    SPI_Init();                // SD card logging
    CAN_Init(125);             // 125 kbps industrial CAN bus

    VICVectAddr0 = (unsigned long)Flame_ISR;
    VICVectCntl0 = 0x20 | 14;  // EINT0 as fire-detect interrupt
    VICIntEnable = (1 << 14);
}
```

This routine configures every peripheral used by the system — GPIO direction registers for LEDs/buzzer/relay, the ADC, UART0 for the HC-05 module, I2C for the RTC and EEPROM, SPI for SD-card logging, and the CAN controller — and installs the flame-sensor interrupt handler on EINT0 so that fire detection does not depend on the polling loop.

(c) Sensor Acquisition

```
void Sensor_Read(void)
{
    DHT11_Read(&temp, &humidity);   // one call returns both values

    gas      = ADC_Read(CH_MQ2);
    vib      = ADC_Read(CH_VIBRATION);
    current  = ADC_Read(CH_ACS712);
}
```

The DHT11 is a digital sensor, so a single DHT11_Read() call over its 1-wire GPIO line returns both temperature and humidity in one transaction — no separate ADC channel is needed for either parameter. The remaining analog sensors (gas, vibration, current) are still sampled through the microcontroller's on-chip ADC. Keeping this as a dedicated function isolates the hardware-specific channel/pin mapping from the decision logic that follows.

(d) Threshold Evaluation and Decision Logic

```
void Evaluate_Conditions(void)
{
    if (temp > TEMP_LIMIT)
        Raise_Alert("HIGH TEMP ALERT", WARNING);

    else if (gas > GAS_LIMIT)
        Raise_Alert("GAS LEAKAGE DETECTED", EMERGENCY);

    else if (current > CURRENT_LIMIT)
        Raise_Alert("OVER CURRENT ALERT", EMERGENCY);

    else if (vib > VIB_LIMIT)
        Raise_Alert("MACHINE VIBRATION ALERT", WARNING);

    else if (humidity > HUMIDITY_LIMIT)
        Raise_Alert("HIGH HUMIDITY DETECTED", WARNING);

    else
    {
        IOSET0 = GREEN_LED; IOCLR0 = RED_LED | BUZZER;
        LCD_Print("SYSTEM NORMAL");
    }
}
```

This function embodies the “AI-inspired” decision-making referred to in the project brief: rather than a machine-learning model, it is a deterministic, priority-ordered rule set that inspects every parameter each cycle and reacts to the most safety-critical condition first (gas and over-current are treated as emergencies before the lower-severity vibration and humidity warnings).

(e) Alert Generation

```
void Raise_Alert(char *msg, unsigned char level)
{
    LCD_Print(msg);
    IOSET0 = RED_LED | BUZZER;

    if (level == EMERGENCY)
        IOCLR0 = RELAY;              // shut down the machine

    UART0_SendString(msg);           // HC-05 Bluetooth alert
    RTC_LogEvent(msg);               // time-stamp via DS1307
    EEPROM_WriteLog(msg);            // persist fault history

    if (level == EMERGENCY)
        CAN_Broadcast(msg);          // notify other plant nodes
}
```

Raise_Alert() centralises every output action — LCD message, LED/buzzer activation, relay control, Bluetooth transmission, RTC time-stamping, EEPROM logging and CAN broadcast — so that any new alert type only needs to call this single function rather than duplicating output-handling code.

(f) Fire Detection Interrupt Service Routine

```
__irq void Flame_ISR(void)
```

```
{
    IOSET0 = RED_LED;
    IOCLR0 = RELAY;                  // immediate machine shutdown
    UART0_SendString("FIRE DETECTED");
    CAN_Broadcast("FIRE DETECTED");
    RTC_LogEvent("FIRE DETECTED");

    EXTINT |= (1 << 0);              // clear external interrupt flag
    VICVectAddr = 0;                 // acknowledge interrupt
}
```

Because fire is the most time-critical hazard, it is handled through a hardware interrupt (EINT0) rather than the polling loop used for the other sensors. This guarantees an immediate response — LED, relay shutdown, Bluetooth alert and CAN broadcast — regardless of what the main loop is doing at that instant.

4.3 Code Analysis and Review Remarks

Review Aspect	Observation
Modularity	Each subsystem (init, sensing, decision, alerting, ISR) is isolated in its own function, which simplifies unit testing and future upgrades.
Readability	Named constants (TEMP_LIMIT, GAS_LIMIT, etc.) replace magic numbers, making the threshold logic self-documenting.
Response Time	Fire detection uses a hardware interrupt rather than polling, ensuring the fastest possible reaction to the highest-risk hazard.
Fault Priority	Evaluate Conditions() checks emergency-level hazards (gas, over-current) before warning-level ones (vibration, humidity), which is appropriate for a safety system.
Data Persistence	Logging to both RTC-time-stamped EEPROM and SD card ensures fault history survives a power interruption.
Scope for Improvement	Threshold values are currently static constants; a future version could store them in EEPROM so they can be re-calibrated without reflashing the firmware.
Scope for Improvement	A watchdog timer is recommended to be added so the controller can recover automatically from an unexpected firmware hang.

CHAPTER 5 | ADVANTAGES AND DISADVANTAGES

5.1 Advantages

- Provides continuous, real-time monitoring of six critical industrial parameters without human intervention.
- Automatic emergency alert generation significantly reduces response time compared to manual inspection.
- Interrupt-based fire detection guarantees an immediate reaction to the most time-critical hazard.
- Relay-based automatic shutdown protects machinery before a fault escalates into serious damage.
- CAN-based communication allows multiple monitoring nodes across a factory to share emergency information instantly.
- RTC and EEPROM-based logging preserves a complete, time-stamped fault history even after power loss, supporting root-cause analysis.
- Bluetooth connectivity allows maintenance personnel to receive alerts directly on a mobile device.
- Modular embedded design combines multiple protocols (GPIO, ADC, UART, I2C, SPI, CAN) in a single industry-oriented platform, giving strong practical learning value.
- Relatively low hardware cost compared to commercial industrial SCADA/monitoring packages.

5.2 Disadvantages / Limitations

- Does not use genuine machine-learning or predictive analytics — decision-making is rule-based (AI-inspired), not true AI.
- Fixed threshold values require manual recalibration for different machines or environments.
- Sensor accuracy (especially MQ2 gas sensor and low-cost vibration sensors) can drift over time and needs periodic calibration.
- Bluetooth (HC-05) has a limited range, making it unsuitable for alerting personnel located far from the monitored machine.
- The prototype is built and tested at a small scale; deployment across a full factory would require additional ruggedisation, EMI shielding and industrial-grade enclosures.
- Single-point sensor failure is not automatically distinguished from a genuine hazard unless additional fault-detection logic is added.

CHAPTER 6 | FUTURE SCOPE

While the current synopsis demonstrates a complete embedded safety-monitoring architecture, several enhancements can extend the system toward a production-ready industrial solution:

- IoT Cloud Integration: Streaming sensor data and alerts to a cloud dashboard for remote, plant-wide monitoring and analytics.
- Mobile Application Control: A dedicated Android/iOS application to receive alerts, view live sensor status and acknowledge emergencies, replacing the limited-range Bluetooth link with Wi-Fi/GSM connectivity.
- AI-Based Predictive Maintenance: Applying genuine machine-learning models on historical sensor logs to predict failures before they occur, rather than only reacting after a threshold is crossed.
- Wireless Industrial Node Communication: Extending the current wired CAN network with wireless mesh communication (e.g. Zigbee/LoRa) for large or hard-to-wire facilities.
- Multi-Machine Monitoring: Scaling the architecture to simultaneously monitor tens or hundreds of machines through a distributed network of nodes.
- Camera-Based Monitoring: Adding vision-based inspection (e.g., smoke/flame image recognition) to complement the existing sensor suite.
- Cloud Report Generation: Automatically compiling logged fault history into periodic maintenance and safety-compliance reports.
- Smart Industrial Dashboard: A centralised web dashboard visualising the real-time status, alerts and history of every monitoring node in the plant.

CHAPTER 7 | BIBLIOGRAPHY

The following references and resources were consulted for the design methodology, component selection and embedded programming concepts used in this synopsis report.

- NXP Semiconductors, “LPC2129/2148 ARM7TDMI-S Microcontroller User Manual,” NXP Semiconductors datasheet documentation.
- Texas Instruments, “LM35 Precision Centigrade Temperature Sensors,” LM35 datasheet.
- Hanwei Electronics, “MQ-2 Semiconductor Sensor for Combustible Gas,” MQ-2 gas sensor datasheet.
- Allegro Microsystems, “ACS712 Fully Integrated, Hall-Effect-Based Linear Current Sensor,” ACS712 datasheet.
- Maxim Integrated / Analog Devices, “DS1307 64 x 8 Serial Real-Time Clock,” DS1307 datasheet.
- Microchip Technology, “24AA256/24LC256/24FC256 256K I2C Serial EEPROM,” 24C256 datasheet.
- Microchip Technology, “MCP2551 High-Speed CAN Transceiver,” MCP2551 datasheet.
- Bosch, “CAN Specification Version 2.0,” Robert Bosch GmbH, controller area network protocol specification.
- Keil – An ARM Company, “µVision IDE and ARM Development Tools Reference Guide.”
- Flash Magic, “In-System Programming Utility for NXP Microcontrollers – User Guide.”
- Muhammad Ali Mazidi et al., “ARM Assembly and C Programming for Embedded System Design,” Pearson Education.
- Steve Furber, “ARM System-on-Chip Architecture,” 2nd Edition, Addison-Wesley.
- Relevant IEEE papers and technical articles on embedded industrial safety monitoring and CAN-based industrial automation systems, referred to for background study and literature support.